INTEGRAL DRAGON

MODULE 2 • SESSION 07

Context Control via CLAUDE.md

Put stable rules in the harness, not in the prompt

AI-Accelerated Software Development | Foundations Program

Confidential. Prepared exclusively for Ensemble Health Partners.

A repeated reminder is not a rule — it is a configuration gap

WHAT TEAMS KEEP DOING

- Typing "use our testing conventions" at the start of every session
- Reminding Claude not to touch protected paths every time
- Repeating the same architectural constraints across engineers
- Hoping the model remembers something it was told three messages ago

The result

Variance across teammates. Output that drifts depending on who prompted. Rules that exist only in someone's memory instead of the project itself.

THE BETTER APPROACH

- Stable rules belong in the environment, not in a one-off prompt
- `CLAUDE.md` carries durable instructions the agent applies on every task
- Project rules become part of the harness, not tribal knowledge
- Versioned in git — reviewed in PRs, shared across the team

Session goal

Know what belongs in `CLAUDE.md`, how scope layers work, and how to write rules that actually change behavior.

CLAUDE.md is shared working context — project memory with rules, not a giant prompt scrapbook

Part of the Environment, Not the Task

CLAUDE.md carries durable instructions the agent applies repeatedly. It is loaded into every session automatically — not because you remembered to paste it, but because it lives in the project.

Durable Instructions for Repeating Patterns

If an instruction applies to more than one task, it belongs here. Architecture boundaries, testing expectations, review requirements, and formatting conventions are all candidates.

Version-Controlled and Team-Shared

Because CLAUDE.md lives in the repository, it travels with the code. Every teammate works from the same rules. Changes go through pull requests. The file earns the same engineering discipline as the code it governs.

What belongs here

- Coding and formatting conventions
- Architecture boundaries and no-go zones
- Testing and verification expectations
- Review and narration requirements
- Repo-specific workflow rules

What does not belong

- Temporary task goals
- One-time business decisions
- Speculative background dumps
- Instructions that change every session

Scope is part of the control model — the right layer solves the right problem

USER

Personal preferences that apply across all your projects. Your default style and workflow habits.

Example: preferred diff narration style, terminal output verbosity

PROJECT

Rules shared with the team, checked into the repository. The most important layer for enterprise consistency.

Example: architecture boundaries, testing requirements, protected paths

LOCAL

Per-machine or experimental overrides. Not shared with teammates. Useful for local exploration.

Example: verbose debug output for your own dev environment

MANAGED

Organization-enforced policy. Cannot be casually bypassed. Applied centrally across the enterprise.

Example: compliance rules, approved model list, mandatory audit logging

Key rule: Team conventions belong in **project scope**. Personal experiments belong in **local scope**. Enterprise policy belongs in **managed settings**. Do not hide shared project rules in your private user file.

A good CLAUDE.md rule is short, enforceable, and worth carrying forward across every task

TOO VAGUE
"Write clean code."

→

ENFORCEABLE
"Prefer existing service abstractions. Do not introduce new top-level folders without approval."

TOO VAGUE
"Be careful with tests."

→

ENFORCEABLE
"Run targeted tests for changed modules and narrate which were run or skipped."

TOO VAGUE
"Don't touch the database."

→

ENFORCEABLE
"No schema changes without explicit approval in the task. Preserve all existing audit log columns."

Bad CLAUDE.md files fail in two ways: too vague to change behavior, or too bloated to stay relevant. The goal is durable, high-signal constraints — few enough to remember, precise enough to enforce.

LIVE DEMO — SESSION 07

Behavior changes when the rules move into the repo

Same task. Same model. Different output — because the environment changed.

Rule in prompt vs. rule in harness — what the difference looks like in practice

Without a project rule

```
$ cd my-service
> "Refactor the payment processor to
    reduce duplication."

# Output varies depending on who asks.
# No narration. Test coverage undefined.
# May touch protected paths.
```

After adding CLAUDE.md

```
# CLAUDE.md in repo root:
# - Prefer existing PaymentService methods.
# - Do not modify /migrations.
# - Run tests for changed modules.
# - Narrate file-level impact.
```

Same request — different behavior

```
> "Refactor the payment processor to
    reduce duplication."

→ Respects PaymentService boundaries.
→ No migration files touched.
→ Reports: "ran payment_test.py;
         skipped db_schema_test.py (not in scope)"
→ Narrates: "modified processor.py L42-58,
          extracted duplicate logic to helper.py"
```

What changed

Nothing in the prompt. Nothing in the model. The project rules moved from someone's memory into the environment — and the behavior followed.

CLAUDE.md complements specs and plans — each layer controls a different thing

The Spec Defines This Task

Machine-readable acceptance criteria for one specific piece of work. What the output should do, how it should be validated, what edge cases apply. Task-scoped by definition.

The Plan Defines This Execution Path

How Claude intends to implement the spec — which files, which changes, in what order. Reviewed and approved before any writes. Session-scoped.

CLAUDE.md Defines Recurring Project Rules

The durable constraints that apply to every task, every session, every engineer on the team. Architecture conventions, testing standards, protected zones. Always in scope.

Hooks and Permissions Enforce Deterministically

The controls that run mechanically regardless of what Claude decided. These sit outside model judgment entirely and provide the final enforcement layer around everything above.

Regulated team examples for CLAUDE.md

- No schema changes without explicit approval
- Preserve existing audit logging columns
- Keep diffs small; narrate file-level impact
- Do not introduce new dependencies without justification

Common ways teams get CLAUDE.md wrong — and what to do instead

Stuffing Every Thought In

A bloated CLAUDE.md becomes background fog. When the file contains 80 lines of loosely related guidance, none of it gets reliable weight. Rules compete for attention. The high-value constraints get buried.

Hiding Project Rules in Local Files

Shared standards that belong in the repo end up in one engineer's personal config. New teammates and CI agents don't see them. The rules exist on one machine and nowhere else.

Encoding Personal Taste as Team Policy

Preferences that belong in user or local scope end up in the project file. Now every teammate's agent is enforcing one engineer's idiosyncratic habits. Scope confusion creates hidden inconsistency.

Forgetting to Update It

CLAUDE.md rules decay. Architectures change, services get renamed, constraints evolve. A stale file enforces yesterday's standards. Treat it like code: review it, update it, remove rules that no longer apply.

The test: Can every rule in your CLAUDE.md survive the question "would this meaningfully change agent behavior on any task this week?" If not, remove it.

SESSION 07 — KEY TAKEAWAY

Stable rules belong in the environment

CLAUDE.md is how teams turn repeated verbal reminders into durable project controls.

Next: Session 08 — Extending the environment outward by connecting Claude to external tools and systems through MCP.